



Adapter Non-Bypassability Architecture

Public Technical Disclosure – Prior Art Publication

Publication Date: January 8, 2026

License: CC0 1.0 Universal (Public Domain Dedication)

Status: Defensive Publication for Prior Art

Abstract

This document discloses an architectural pattern where tool execution is gated by mandatory capability token verification at the adapter layer, preventing unauthorized actions **even if the policy engine, orchestrator, or AI model is compromised**. Adapters serve as the single enforcement boundary between intent (what an agent wants to do) and action (what actually executes). The system fails closed—no token, no action.^[1]

This specification is published as prior art to establish the adapter-enforced verification pattern for AI agent authorization systems and prevent patent enclosure of verification gates at tool boundaries.^[2]

1. The Core Security Problem

In autonomous agent systems, three attack surfaces exist:

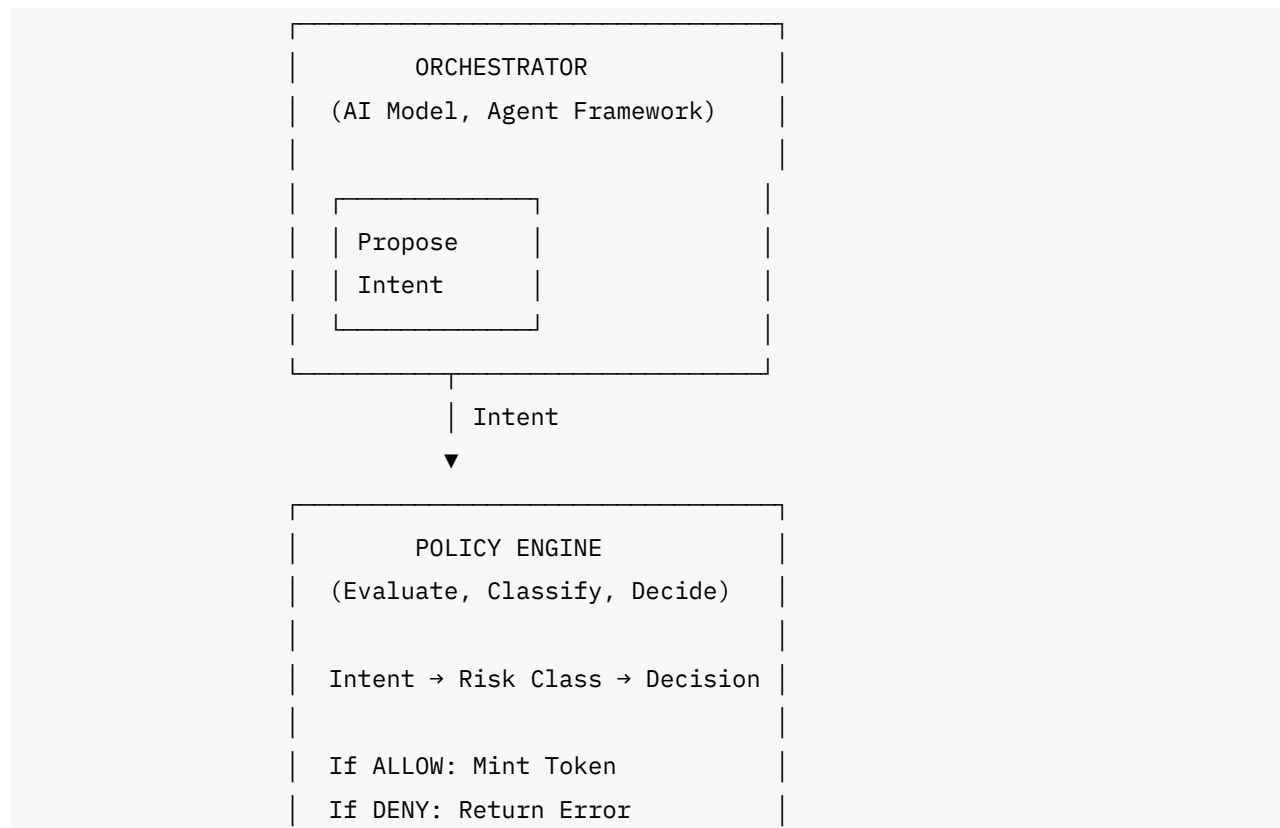
1. **Model/Orchestrator Compromise:** Prompt injection, jailbreaks, adversarial inputs cause the AI to request unauthorized actions
2. **Policy Engine Bypass:** Attacker circumvents policy evaluation entirely
3. **Tool Credential Theft:** Attacker obtains tool credentials and calls tools directly

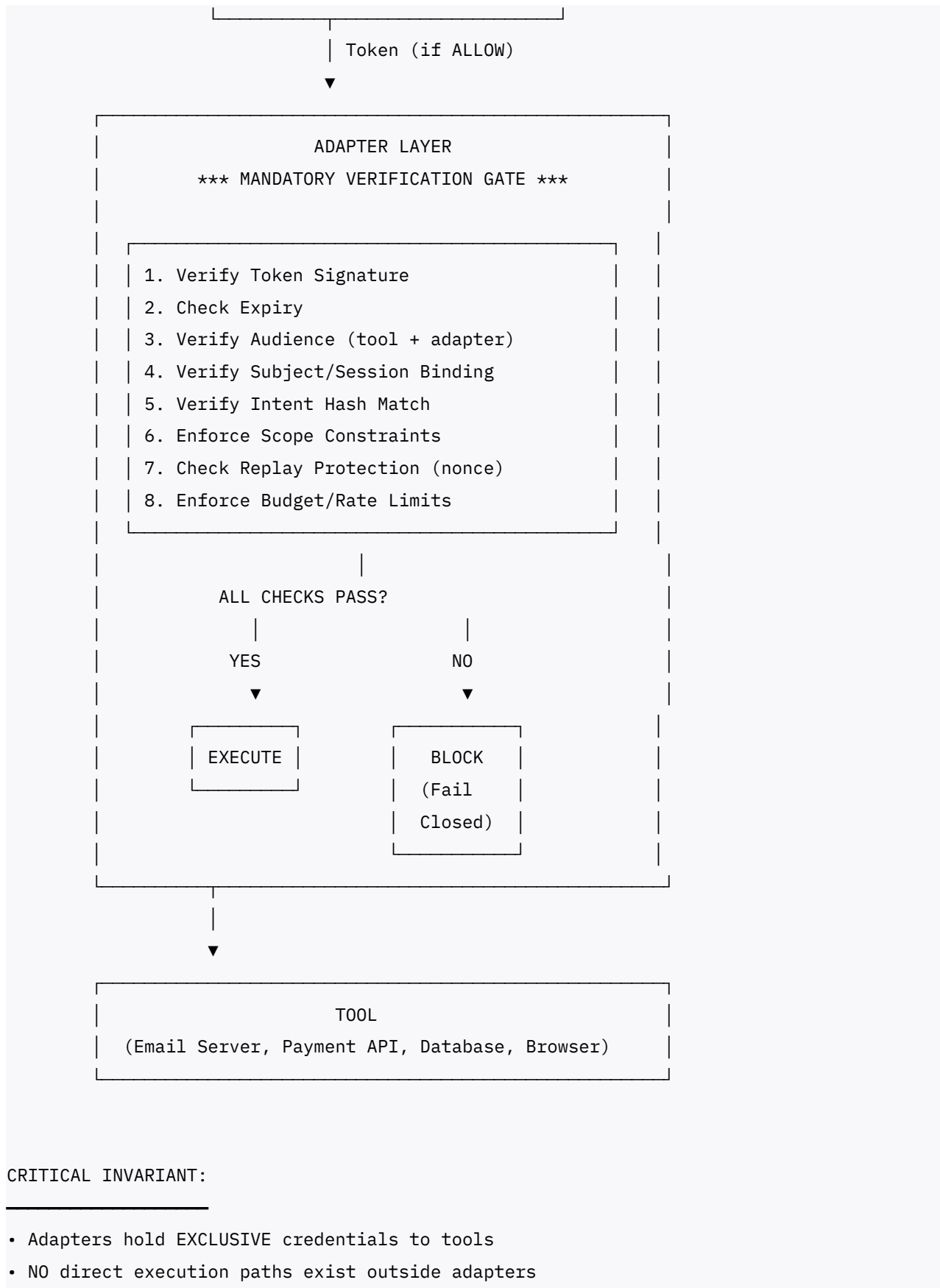
Traditional architectures fail because:

- The orchestrator holds tool credentials
- Policy checks occur in-band (same trust domain as model)
- No cryptographic binding between authorization and execution

The Solution: Place enforcement at the **point of effect**—the adapter that interfaces with the tool.

2. Architectural Principle: Single Execution Path





- Orchestrator layer CANNOT bypass token verification
- Even if orchestrator is compromised, tools require valid token

3. Fail-Closed Enforcement

The adapter operates on a fail-closed principle:^[1]

DEFAULT BEHAVIOR: DENY

```
If token is missing      → BLOCK
If signature is invalid  → BLOCK
If token is expired      → BLOCK
If audience is wrong     → BLOCK
If intent hash mismatches → BLOCK
If nonce is replayed     → BLOCK
If budget is exhausted   → BLOCK
If any check fails       → BLOCK
```

```
ONLY if ALL checks pass  → EXECUTE
```

No exception paths exist. The adapter has no "override" mode, no "debug" flag, no "admin bypass." Either the token is valid and all bindings match, or the action does not execute.

4. Adapter Interface Specification

Every adapter implements the following interface:^[1]

```
interface LambdaProofAdapter {
  // Adapter identity
  readonly adapterId: string;    // e.g., "email_adapter_v1"
  readonly toolName: string;     // e.g., "email"
  readonly version: string;      // e.g., "1.0.0"

  // Lifecycle
  initialize(config: AdapterConfig): Promise<void>;
  shutdown(): Promise<void>;
}
```

```

// Core enforcement (THE CRITICAL PATH)
enforceAction(
    token: CapabilityToken,
    action: ToolAction
): Promise<EnforcementResult>;

// Token validation (called by enforceAction)
validateToken(token: CapabilityToken): Promise<TokenValidation>;

// Audit emission (called after every decision)
emitAuditEvent(event: AdapterAuditEvent): Promise<string>;

// Health status
getStatus(): AdapterStatus;
}

```

4.1 Configuration

```

interface AdapterConfig {
    // Policy engine connection
    policyEndpoint: string;

    // Archivum connection for audit receipts
    archivumEndpoint: string;

    // Cryptographic material
    adapterSigningKey: string;           // For signing audit events
    tokenVerificationKey: string;        // Public key for token verification

    // Operational parameters
    enforcementMode: "strict" | "permissive" | "audit-only";
    silentModeEnabled: boolean;          // For drift/CSL failures
    maxDriftThreshold: number;           // Default: 0.3

    // Tool-specific configuration
    toolConfig: Record<string, unknown>;
}

```

4.2 Enforcement Result

```
interface EnforcementResult {
    status: "EXECUTED" | "BLOCKED" | "SILENT";

    // Execution details (if EXECUTED)
    executionId?: string;
    toolResponse?: unknown;

    // Blocking reason (if BLOCKED or SILENT)
    reason?: BlockReason;

    // Audit reference (ALWAYS present)
    auditEventId: string;
    archivumCid: string;

    // Timing
    enforcementTimestamp: number;
    executionDuration?: number;
}

type BlockReason =
    | "TOKEN_INVALID"
    | "TOKEN_EXPIRED"
    | "TOKEN_USED"
    | "DRIFT_EXCEEDED"
    | "PRIME_GATE_CLOSED"
    | "CSL_VIOLATION"
    | "INTENT_HASH_MISMATCH"
    | "AUDIENCE_MISMATCH"
    | "SCOPE_VIOLATION"
    | "BUDGET_EXHAUSTED"
    | "NONCE_REPLAYED"
    | "TOOL_ERROR";
```

5. Verification Checklist (Normative)

An adapter MUST verify the following, in this order:^{[21][1]}

#	CHECK
1	TOKEN SIGNATURE • Verify signature under issuer key (identified by key_id) • Reject if signature invalid • Error: ACI-001 TOKEN_SIGNATURE_INVALID
2	TEMPORAL VALIDITY • Token.notBefore ≤ current_time + CLOCK_SKEW (30s default) • Token.notAfter ≥ current_time - CLOCK_SKEW • Error: ACI-002 TOKEN_EXPIRED or ACI-003 TOKEN_NOT_YET_VALID
3	AUDIENCE BINDING • Token.audience.tool == this adapter's toolName • Token.audience.adapter_id == this adapter's adapterId • Error: (no code - silent rejection)
4	SUBJECT/SESSION BINDING • Token.subject.subject_id matches request context • Token.subject.session_id matches current session (if set) • Optional: verify client_pubkey binding • Error: (no code - silent rejection)
5	INTENT HASH VERIFICATION • Reconstruct canonical intent from action request • Compute intent_hash = SHA256(canonical_bytes) • Compare with token.binding.intent_hash • Error: ACI-005 INTENT_HASH_MISMATCH
6	SCOPE ENFORCEMENT • Requested action ∈ token.scope.actions • All params satisfy token.scope.constraints • amount ≤ max_amount, recipient ∈ allowlist, etc. • Error: (scope violation - blocked)
7	REPLAY PROTECTION • If token.replay_protection.one_time == true: - Check nonce against replay cache - If found: reject (ACI-004 TOKEN_USES_EXHAUSTED)

	- If not found: atomically add to cache, proceed • Track token.usedCount against token.maxUses
8	BUDGET/RATE LIMIT ENFORCEMENT • DO NOT trust upstream budget tracking • Locally enforce token.limits.rate_limit • Locally enforce token.limits.budget • Decrement budget atomically per execution
9	DRIFT & PRIME GATE (Optional - Δ Proof specific) • Compute drift score from token metadata • If drift > maxDriftThreshold: enter SILENT mode • If prime gate closed: enter SILENT mode • Error: ACI-006 DRIFT_EXCEEDED, ACI-007 PRIME_GATE_CLOSED

IF ALL CHECKS PASS → Execute action, emit EXECUTED audit event

IF ANY CHECK FAILS → Block action, emit BLOCKED audit event

IF DRIFT/CSL FAILS → Silent mode, emit SILENT audit event (archived only)

6. Silent Mode: Fail-Safe for Ethical Violations

When drift exceeds bounds or CSL (Conscious Sovereignty Layer) checks fail, the adapter enters **silent mode**:^[1]

SILENT MODE BEHAVIOR:

- Action is NOT executed
- NO error is returned to the agent
- Audit event IS recorded with reason
- Receipt IS archived to Δ -Archivum
- System appears to "do nothing" gracefully

PURPOSE:

- Prevents adversarial probing ("what triggers CSL?")
- Maintains ethical invariants without disclosure

- Aligned with Silence Clause: "when in doubt, do nothing"

7. Credential Isolation: Why Adapters Cannot Be Bypassed

The architecture ensures adapters are the **only** path to tool execution:



ATTACK SCENARIOS DEFEATED:

1. Model compromise (prompt injection)
 - Model can propose malicious intents
 - Policy engine may deny OR issue restricted token
 - Adapter STILL verifies all bindings
 - Attack fails at intent_hash mismatch or scope violation
2. Policy engine compromise
 - Attacker can issue arbitrary tokens
 - BUT tokens must be signed with issuer key
 - AND adapter verifies audience + subject + intent
 - Attack fails unless attacker has signing keys
3. Credential theft (orchestrator level)
 - Orchestrator has NO tool credentials
 - Only adapter has tool credentials
 - Attack yields nothing useful
4. Direct tool access attempt
 - Tools are network-isolated
 - Only adapters can reach tools
 - Firewall rules enforce this

8. Reference Implementations

8.1 Web Navigation Adapter

```
class WebNavigationAdapter implements LambdaProofAdapter {  
    readonly adapterId = "web_navigation_v1";  
    readonly toolName = "browser";  
    readonly version = "1.0.0";  
  
    private browser: Browser;
```

```

private config: AdapterConfig;

async enforceAction(
  token: CapabilityToken,
  action: ToolAction
): Promise<EnforcementResult> {
  // STEP 1: Validate token (all 8 checks)
  const validation = await this.validateToken(token);
  if (!validation.valid) {
    return this.blocked(token, validation.reason);
  }

  // STEP 2: Check drift and prime gate
  const drift = await this.computeDrift(token);
  if (drift > this.config.maxDriftThreshold) {
    return this.silent(token, "DRIFT_EXCEEDED");
  }

  // STEP 3: Verify intent hash matches action
  const intentHash = sha256(canonicalize(action));
  if (intentHash !== token.intentHash) {
    return this.blocked(token, "INTENT_HASH_MISMATCH");
  }

  // STEP 4: Enforce scope constraints
  const url = action.params.url as string;
  if (this.isDeniedDomain(url)) {
    return this.blocked(token, "SCOPE_VIOLATION");
  }

  // STEP 5: Execute navigation
  const response = await this.browser.navigate(url);

  // STEP 6: Emit audit event
  const auditId = await this.emitAuditEvent({
    eventType: "ENFORCEMENT",
    decision: "EXECUTED",
    tokenId: token.tokenId,
    executionSuccess: response.ok
  });
}

```

```

    return {
      status: "EXECUTED",
      executionId: response.requestId,
      toolResponse: response.content,
      auditEventId: auditId,
      archivumCid: response.archivumCid,
      enforcementTimestamp: Date.now()
    };
  }
}

```

8.2 Payment Transfer Adapter

```

class PaymentTransferAdapter implements LambdaProofAdapter {
  readonly adapterId = "payment_transfer_v1";
  readonly toolName = "payments";
  readonly version = "1.0.0";

  async enforceAction(
    token: CapabilityToken,
    action: ToolAction
  ): Promise<EnforcementResult> {
    // HIGHER SCRUTINY: Financial actions require ZK proof level
    if (token.proofLevel !== "ZK") {
      return this.blocked(token, "INSUFFICIENT_PROOF_LEVEL");
    }

    // STEP 1: Full token validation
    const validation = await this.validateToken(token);
    if (!validation.valid) {
      return this.blocked(token, validation.reason);
    }

    // STEP 2: Verify ZK proof (if required)
    if (token.proofLevel === "ZK") {
      const zkValid = await this.verifyZKProof(token.zkProof);
      if (!zkValid) {
        return this.blocked(token, "ZK_PROOF_INVALID");
      }
    }
  }
}

```

```

    }
}

// STEP 3: Enforce amount constraints
const amount = action.params.amount as number;
const maxAmount = token.scope.constraints.amount_max;
if (amount > maxAmount) {
    return this.blocked(token, "SCOPE_VIOLATION");
}

// STEP 4: Verify beneficiary is in allowlist
const beneficiary = action.params.beneficiary_id as string;
const allowedBeneficiaries = token.scope.constraints.beneficiary_allowlist;
if (!allowedBeneficiaries.includes(beneficiary)) {
    return this.blocked(token, "SCOPE_VIOLATION");
}

// STEP 5: Execute transfer
const transfer = await this.executeTransfer({
    amount: action.params.amount,
    recipient: action.params.recipient,
    memo: action.params.memo
});

// STEP 6: Emit audit event
const auditId = await this.emitAuditEvent({
    eventType: "ENFORCEMENT",
    decision: "EXECUTED",
    tokenId: token.tokenId,
    executionSuccess: transfer.success
});

return {
    status: "EXECUTED",
    executionId: transfer.transactionId,
    auditEventId: auditId,
    archivumCid: transfer.archivumCid,
    enforcementTimestamp: Date.now()
};
}

```

```
}
```

9. Audit Event Schema

Every enforcement decision produces an audit event:^[1]

```
interface AdapterAuditEvent {
    // Event identity
    eventId: string;           // Unique identifier
    eventType: "ENFORCEMENT";

    // Context
    adapterId: string;
    tokenId: string;
    intentHash: string;

    // Decision
    decision: "EXECUTED" | "BLOCKED" | "SILENT";
    reason?: BlockReason;

    // Timing
    timestamp: number;
    primeIndex: number;       // Prime epoch at decision time

    // Drift metrics (no PII)
    driftScore?: number;
    driftThreshold: number;

    // Tool response summary (no PII)
    executionSuccess?: boolean;
    executionDuration?: number;

    // Cryptographic binding (hash chain)
    previousEventHash: string; // Chain linkage
    eventHash: string;         // Poseidon(event_fields)
    signature: string;          // Adapter signature
}
```

Privacy Guarantee: Audit events contain only hashed commitments and operational metadata. No user data, intent parameters, or tool responses are included unless explicitly consented.^[1]

10. Error Codes

Code	Name	Description
ACI-001	TOKEN_SIGNATURE_INVALID	Token signature verification failed
ACI-002	TOKEN_EXPIRED	Token notAfter timestamp exceeded
ACI-003	TOKEN_NOT_YET_VALID	Token notBefore timestamp not reached
ACI-004	TOKEN_USES_EXHAUSTED	Token maxUses exceeded
ACI-005	INTENT_HASH_MISMATCH	Action params don't match intent hash
ACI-006	DRIFT_EXCEEDED	Drift score above threshold
ACI-007	PRIME_GATE_CLOSED	Current prime epoch gate is closed
ACI-008	CSL_VIOLATION	CSL commutation check failed
ACI-009	TOOL_UNAVAILABLE	Underlying tool not responding
ACI-010	ADAPTER_NOT_INITIALIZED	Adapter not properly initialized

11. Conformance Requirements

11.1 Adapters MUST:

1. Implement the full `LambdaProofAdapter` interface
2. Validate token signatures before any enforcement decision
3. Check drift against configured threshold
4. Emit audit events for all enforcement decisions
5. Enter silent mode when drift exceeds bounds or prime gate is closed

6. **Never execute actions without valid tokens (fail-closed)**
7. Never log or transmit PII in audit events

11.2 Adapters MAY:

- Implement tool-specific extensions to the base interface
- Add additional validation steps before enforcement
- Support multiple tool actions within a single adapter

11.3 Adapters MUST NOT:

- Execute actions when tokens are invalid, expired, or exhausted
 - Expose policy evaluation logic or risk classification details
 - Store raw intent parameters or user data beyond hashed commitments
 - Provide any bypass mechanism (debug mode, admin override, etc.)
-

12. Threat Model & Security Properties

12.1 Threats Mitigated

Threat	Mitigation
Prompt injection	Adapter verifies intent_hash; malicious params produce different hash → BLOCKED
Model jailbreak	Model can request anything; adapter enforces scope constraints → BLOCKED
Policy engine bypass	Tokens require valid signature; adapter verifies → BLOCKED
Token theft	Subject binding + session binding → Token unusable by attacker
Replay attacks	Nonce tracking + one-time tokens → BLOCKED
Parameter tampering	Intent hash binding → BLOCKED

Orchestrator compromise	Orchestrator has no tool credentials → Attack yields nothing
Direct tool access	Network isolation; tools only accessible via adapters

12.2 Residual Risks

Risk	Mitigation Strategy
Adapter compromise	HSM key storage, code signing, minimal attack surface
Signing key theft	HSM/KMS, key rotation, dual-control issuance
Side-channel attacks	Constant-time verification, timing isolation
Collusion (adapter + policy engine)	Separation of duties, audit chain verification

13. Prior Art Statement

This specification is published as prior art under **CC0 1.0 Universal Public Domain Dedication** to:

1. **Establish prior art** for the adapter-enforced verification pattern
2. **Prevent patent enclosure** of verification gates at tool boundaries
3. **Enable interoperability** across agent authorization systems
4. **Block trivial patent claims** on mandatory verification, fail-closed enforcement, and credential isolation

What this document covers (PUBLIC):

- Adapter interface specification
- Verification checklist (all 9 checks)
- Credential isolation architecture
- Silent mode behavior
- Audit event schema
- Reference implementations

- Conformance requirements

What remains proprietary (NOT DISCLOSED):

- Load balancing algorithms across adapters
- Failover strategies (cascade vs. retry vs. abort)
- Replay cache implementation (data structures, eviction)
- Verification result caching strategies
- Drift calculation formulas
- Prime gate synchronization protocol

14. Comparison with Related Work

System	Approach	Key Difference
Adapter Non-Bypassability	Mandatory verification at tool boundary	Cryptographic binding + fail-closed + credential isolation
OAuth Resource Servers	Token verification at API gateway	Coarse-grained; no intent binding; orchestrator holds credentials
API Gateways (Kong, etc.)	Rate limiting + auth at edge	No cryptographic action binding; policy in-band
Zero Trust Network Access	Continuous verification	Network-level; not action-level
Capability-based Security (POLA)	Principle of least authority	Conceptual; this spec provides concrete protocol

Key Novelty:

The combination of **credential isolation** (adapters hold exclusive tool access) + **fail-closed enforcement** (no token = no action) + **intent hash verification** (cryptographic binding to specific action) creates a non-bypassable gate even when upstream components are compromised.

15. Revision History

Version	Date	Changes
1.0	January 8, 2026	Initial specification

End of Specification

How does this look? Would you like me to:

1. **Add network topology diagrams** showing firewall rules and isolation boundaries?
2. **Expand the reference implementations** with database and code runner adapters?
3. **Add formal security proofs** for the non-bypassability claims?
4. **Create conformance test vectors** for adapter certification?
5. **Proceed to the fourth document** (Audit Event Chain with Merkle Checkpoints)?

**

-
1. [Adapter-Coordination-Interface-Specification.pdf](#)
 2. [lets-discuss-licensing-open-so-HV_bbj5uQIOPuXCF7b61YQ.md](#)
 3. [Audit Logs_ Event Schema & Sanitized Examples.pdf](#)
 4. [ΛProof Intent Schema Specification.pdf](#)
 5. [Conscious Sovereignty Layer \(CSL\) Solver.pdf](#)
 6. [ΛProof Protocols.pdf](#)
 7. [DAO Charter.pdf](#)
 8. [Building a ΛDapp_ A Developer's Implementation Guide.pdf](#)
 9. [Capability Token Semantic Mapping.pdf](#)
 10. [Data Sovereignty Informational Personhood.pdf](#)
 11. [ΛProof_ An Architectural Blueprint for Verifiable Artificial Intelligence.pdf](#)

12. [Λ^P-Archivum.pdf](#)
13. [ΛProof - R96 Credits.pdf](#)
14. [Governance for a Non-Surveillance Digital Ecosystem.pdf](#)
15. [Token Specification.pdf](#)
16. [Risk Classification.pdf](#)
17. [Policy Rules Specification.pdf](#)
18. [Evidence Validation.pdf](#)
19. [How Trust is Measured.pdf](#)
20. [ΛProof Token Issuance.pdf](#)